UNIT- III Raspberry Pi with Python Programming for IoT

- **Introduction to Python Programming**
- **Basics Overview of Raspberry Pi and Its Role in IoT,**
- **Interfacing Raspberry Pi with Sensors and Actuators (UART, SPI, I2C).**
- **Data Acquisition and Local Processing on Raspberry Pi,**
- **Sending Data to Cloud Platforms.**
- **Implementation of IoT Projects Using Raspberry Pi**
- **Case Studies:**
  - **Smart Home Automation,**
  - **Healthcare Monitoring,**
  - **Environmental Sensing**

What Is Python?

- **Python** is a general purpose, high-level, interpreted programming language.

- Developed by **Guido van Rossum** in the late 1980s at the National Research Institute for Mathematics and Computer Science in the Netherlands.



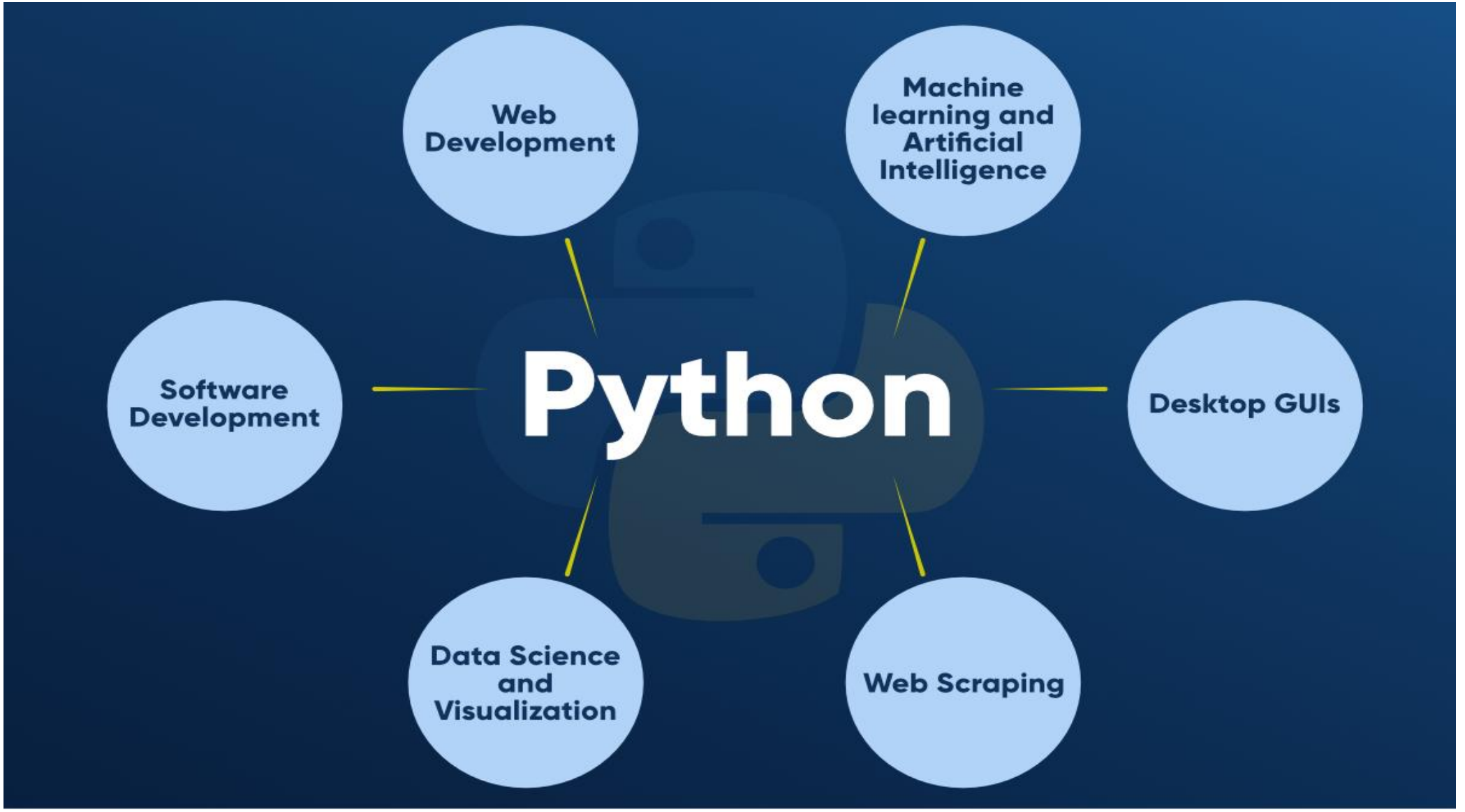
- **Python** is one of the most popular and widely used programming language used for set of tasks including console based, GUI based, web programming and data analysis.

Fact: **Python** is named based on the comedy television show *Monty Python's Flying Circus*.

Python Features

- **Easy to Learn and Use**
- **Interpreted Language**
- **Cross-platform Language**
- **Free and Open Source**
- **Object-Oriented Language**
- **GUI Programming Support**
- **Integrated**

Python Applications



Basics of Python

- Comments in Python
- Variable Declaration in Python
- Data Types in Python
- Type Conversion in Python
- Operators in Python

Comments in Python

- In general, Comments are used in a programming language to describe the program or to hide the some part of code from the interpreter.

- Comment is not a part of the program, but it enhances the interactivity of the program and makes the program readable.

Python supports two types of comments:

- Single Line Comment
- Multi Line Comment

Comments in Python

Cont..

Single Line Comment:

In case user wants to specify a single line comment, then comment must start with **'#'**

Example:

```
# This is single line comment
print "Hello Python"
```

Output:

Hello Python

Multi Line Comment:

Multi lined comment can be given inside triple quotes.

Example:

```
'''This is
   Multiline
   Comment'''
print "Hello Python"
```

Output:

Hello Python

Variable Declaration in Python

- A variable is a named memory location in which we can store values for the particular program.
- Variable also known as identifier and used to hold value.

- In Python, We don't need to declare explicitly variable in Python. When we assign any value to the variable that variable is declared automatically.

- In Python, We don't need to specify the type of variable because Python is a loosely typed language.

Variable Declaration in Python Cont..

- The variable automatically changes it's data type based on assigned value.

Rules for naming variable:

- Variable names can be a group of both letters and digits, but they have to begin with a letter or an underscore.
- It is recommended to use lowercase letters for variable name. ‘SUM’ and ‘sum’ both are two different variables.

Example: Vardemo.py

```
a=10 #integer
b="StudyGlance" #string
c=12.5 #float
print(a)
print(b)
print(c)
```

output:

```
$python3 Vardemo.py
10
StudyGlance
12.5
```

Variable Declaration in Python Cont..

- Python allows us to assign a value to multiple variables and multiple values to multiple variables in a single statement which is also known as multiple assignment.
- **Assign single value to multiple variables :**

Example: Vardemo1.py

```
x=y=z=50
print x
print y
print z
```

output:

```
$python3 Vardemo1.py
50
50
50
```

- **Assign multiple values to multiple variables :**

Example: Vardemo2.py

```
a,b,c=5,10,15
print a
print b
print c
```

output:

```
$python3 Vardemo2.py
5
10
15
```

Data Types in Python

- In general, Data Types specifies what type of data will be stored in variables. Variables can hold values of different data types.

- Python is a dynamically typed or loosely typed language, hence we need not define the type of the variable while declaring it.

- The interpreter implicitly binds the value with its type.

- Python provides us the **type ()** function which enables us to check the type of the variable.

Data Types in Python

Cont..

- Python provides following **standard data types**, those are
  - ✓**Numbers**
  - ✓**String**

Numbers:

- Number stores numeric values. Python creates Number type variable when a number is assigned to a variable.

There are three numeric types in Python:

1. int
2. float
3. Complex

Data Types in Python

Cont..

1. int:

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

Example:

a=10
b=-12
c=123456789

2. float:

Float or "floating point number" is a number, positive or negative, containing one or more decimals.

Example:

X=1.0
Y=12.3
Z=-13.4

3. complex:

Complex numbers are written with a "j" as the imaginary part.

Example:

A=2+5j
B=-3+4j
C=-6j

Data Types in Python

Cont..

String:

- The string can be defined as the sequence of characters represented in the quotation marks. In python, we can use single, double, or triple quotes to define a string.

- In the case of string handling, the operator + is used to concatenate two strings as the operation *"hello"+" python"* returns *"hello python"*.

Example:

```
S1='Welcome'  #using single quotes
S2="To"       #using double quotes
S3='''Python'''  #using triple quotes
```

Data Types in Python

Cont..

Example: “datatypesdemo.py”

```
a=10
b=“Python"
c = 10.5
d=2.14j
print("Data type of Variable a :",type(a))
print("Data type of Variable b :",type(b))
print("Data type of Variable c :",type(c))
print("Data type of Variable d :",type(d))
```

Output:

python3 datatypesdemo.py

Datatype of Variable a : <class ‘int’>

Datatype of Variable b : <class ‘str’>

Datatype of Variable c : <class ‘float’>

Datatype of Variable d : <class ‘complex’>

Type Conversion in Python

- Python provides Explicit type conversion functions to directly convert one data type to another. It is also called as **Type Casting** in Python
- Python supports following functions
  1. **int ()** : This function converts **any data type to integer.**
  2. **float()** : This function is used to convert **any data type to a floating point number.**
  3. **str()** : This function is used to convert **any data type to a string.**

Example:"Typeconversiondemo.py"

```
x = int(2.8)
y = int("3")
z = float(2)
s = str(10)
print(x);print(y)
print(z); print(s)
```

Output:

python3 typeconversiondemo.py

```
2
3
2
10
```

Operators in Python

- The operator can be defined as a symbol which is responsible for a particular operation between two operands.
- Python provides a variety of operators described as follows.

Arithmetic operators :

+ (addition)	**eg:** a=20; b=10 then a **+** b=30
- (subtraction)	**eg:** a=20; b=10 then a **-** b=10
***(multiplication)	**eg:** a=20; b=10 then a * b=200
/ (divide)	**eg:** a=20; b=10 then a **/** b=2
%(remainder)	**eg:** a=20; b=10 then a **%** b=0
// (floor division)	**eg:** a=24; b=7 then a **//** b=3
** (exponent)	**eg:** a=2; b=3 then a ** b=8

Membership operators :

in (True, If the value is present in the data structure)

not in (True, If the value is not present in the data structure)

Operators in Python

Cont...

Comparison operators :

== (Equal to)
!= (Not equal to)
<= (Less than or equal)
>= (Greater than or equal)
< (Less than)
> (Greater than)

Bitwise operators :

& (binary and)
| (binary or)
^ (binary xor)
~ (negation)
<< (left shift)
>> (right shift)

Assignment operators :

= (Assigns to)
+= (Assignment after Addition)
-= (Assignment after Subtraction)
***=** (Assignment after Multiplication)
/= (Assignment after Division)
%= (Assignment after Modulus)
****=** (Assignment after Exponent)
//= (Assignment after floor division)

Logical operators :

and (logical and)
or (logical or)
not (logical not)

Identity operators :

is (Returns true if both variables are the same object)

is not (Returns true if both variables are not the same object)

Indentation in Python

- In Python, indentation is used to declare a block. If two statements are at the same indentation level, then they are the part of the same block.
- For the ease of programming and to achieve simplicity, python doesn't allow the use of curly braces or parentheses for the block level code.
- Indentation is the most used part of the python programming language.
- Generally, a tab space or four spaces are given to indent the statements in python.

Conditional Statements in Python

- Conditional Statements performs different computations or actions depending on conditions.
- In python, the following are conditional statements
  - if
  - if –else
  - if – elif –else

If statement:

- The if statement is used to test a specific condition. If the condition is true, a block of code (if-block) will be executed.

Syntax:

```
if condition:
        statement1
        statement2
```

Conditional Statements in Python

Cont..

Example: **ifdemo.py**

```
a = 33
b = 200
if b > a:
  print ("b is greater than a")
  print ("done…")
```

Output:

python ifdemo.py
b is greater than a
done…

Remember:

input () function is used to get input from user.

Example:

a=input (“Enter a value”)

Conditional Statements in Python

Cont..

If-else statement:

- The if-else statement provides an else block combined with the if statement which is executed in the false case of the condition.

Syntax:

if condition:

    #block of statements

else:

    #another block of statements (else-block)

Output:

python ifelsedemo.py
Enter your age: 19
You are eligible to vote!!

Example: **ifelsedemo.py**

```
age = int(input("Enter your age : "))
if age>=18:
        print("You are eligible to vote !!")
else:
        print("Sorry! you have to wait !!"))
```

Conditional Statements in Python Cont..

If-elif-else statement:

- The elif statement enables us to check multiple conditions and execute the specific block of statements depending upon the true condition among them.

Syntax:

```
if condition1:
    # block of statements
elif condition2:
    # block of statements
elif condition3:
    # block of statements
else:
    # block of statements
```

Conditional Statements in Python

Cont..

Example: **maxnum.py**

```
a=int(input("Enter a value : "))
b=int(input("Enter b value : "))
c=int(input("Enter c value : "))
if (a>b) and (a>c):
        print("Maximum value is :",a)
elif (b>a) and (b>c):
        print("Maximum value is :",b)
else:
        print("Maximum value is :",c)
```

Output:

python maxnum.py
Enter a value: 10
Enter b value: 14
Enter c value: 9
Maximum value is: 14

Loop Statements in Python

- Sometimes we may need to alter the flow of the program. If the execution of a specific code may need to be repeated several numbers of times then we can go for loop statements.
- In python, the following are loop statements
  - while loop
  - for loop

while loop:

- With the while loop we can execute a set of statements as long as a condition is true. The while loop is mostly used in the case where the number of iterations is not known in advance.

Syntax:

```
while expression:
        Statement(s)
```

Loop Statements in Python

Cont..

Example: **whiledemo.py**

```
i=1;
while i<=3:
        print(i);
        i=i+1;
```

Output:

python whiledemo.py

1

2

3

Using else with while loop

- Python enables us to use the while loop with the else block also. The else block is executed when the condition given in the while statement becomes false.

Example: **wedemo.py**

```
i=1;
while i<=3:
        print(i);
        i=i+1;
else:
        print("while loop terminated")
```

Output:

python wedemo.py

1

2

3

while loop terminated

Loop Statements in Python

Cont..

for loop:

- The for loop in Python is used to iterate the statements or a part of the program several times. It is frequently used to traverse the data structures like list, tuple, or dictionary.

Syntax:

for iterating_var **in** sequence:

statement(s)

Example: **fordemo.py**

```
i=1
n=int(input("Enter n value : "))
for i in range(i,n+1):
        print(i,end = ' ')
```

Output:

python fordemo.py

Enter n value: 5

1 2 3 4 5

Loop Statements in Python

Cont..

Using else with for loop

- Python allows us to use the else statement with the for loop which can be executed only when all the iterations are exhausted.
- Here, we must notice that if the loop contains any of the break statement then the else statement will not be executed.

Example: **fedemo.py**

```
for i in range(1,5):
        print(i,end=' ')
else:
        print("for loop completely exhausted");
```

Output:

python fedemo.py

1 2 3 4

for loop completely exhausted

Jump Statements in Python

- Jump statements in python are used to alter the flow of a loop like you want to skip a part of a loop or terminate a loop.
- In python, the following are jump statements
  - break
  - continue

break:

- The break is a keyword in python which is used to bring the program control out of the loop.
- The break statement breaks the loops one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops.
- The break is commonly used in the cases where we need to break the loop for a given condition.

Syntax: break

Jump Statements in Python

Cont..

Example: **breakdemo.py**

```
i = 1
while i < 6:
        print(i)
        if i == 3:
                break
        i += 1
```

Output:

python breakdemo.py

1

2

3

continue:

- The continue statement in python is used to bring the program control to the beginning of the loop.
- The continue statement skips the remaining lines of code inside the loop and start with the next iteration.
- It is mainly used for a particular condition inside the loop so that we can skip some specific code for a particular condition.

Syntax: continue

Jump Statements in Python

Cont..

Example: **continuedemo.py**

```
str =input("Enter any String : ")
for i in str:
        if i == 'h':
                continue;
        print(i,end=" ");
```

Output:

python continuedemo.py
Enter any String : python
p y t o n

Image Read ,Write Operations

- Python supports PIL library for image related operations
- Reading Image in Python:
- PIL: Python Image Library is used to work with image files
- **from PIL import Image**
- Open an image file
- **image=Image.open(image_name)**
- Display the image
- **image.show()**
- Resize(): Resizes the image to the specified size

- **image.resize(255,255)**
- Rotate(): Rotates the image to the specified degrees, counter clockwise
- **image.rotate(90)**
- **Format: Gives the format of the image**
- Size: **Gives a tuple with 2 values as width and height of the image, in pixels**
- **Mode: Gives the band of the image, 'L' for grey scale, 'RGB' for true colour image**
- print(image.format, image.size, image.mode)

- **conv_image=image.convert('L')**
- Conversion between modes other that 'L' and 'RGB' needs conversion into any of these 2 intermediate mode

Output

Converting a sample image to Grey Scale

```
from PIL import Image

im = Image.open('/home/saswati/VRP_Linux/Images/i3.jpg')
im.show()
grey_image=im.convert('L')
grey_image.show()
grey_image.save('GreyScaleImage.jpg')
```

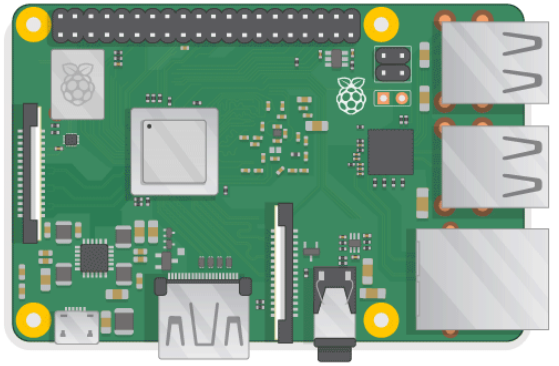
Output





Introduction to Raspberry Pi

What is Raspberry PI ?



Raspberry Pi is a
low-cost
credit-card sized
single-board computer
Developed by **Raspberry Pi Foundation (UK)**
launched in **2012**

Introduction to Raspberry Pi



Why the name- Raspberry Pi

The **name** Raspberry Pi is **derived from the fruit , raspberry** .

This is **because many companies** in the computer neighborhood where Raspberry Pi was based used fruit names **such as Apple and apricot** as names for their companies and products.

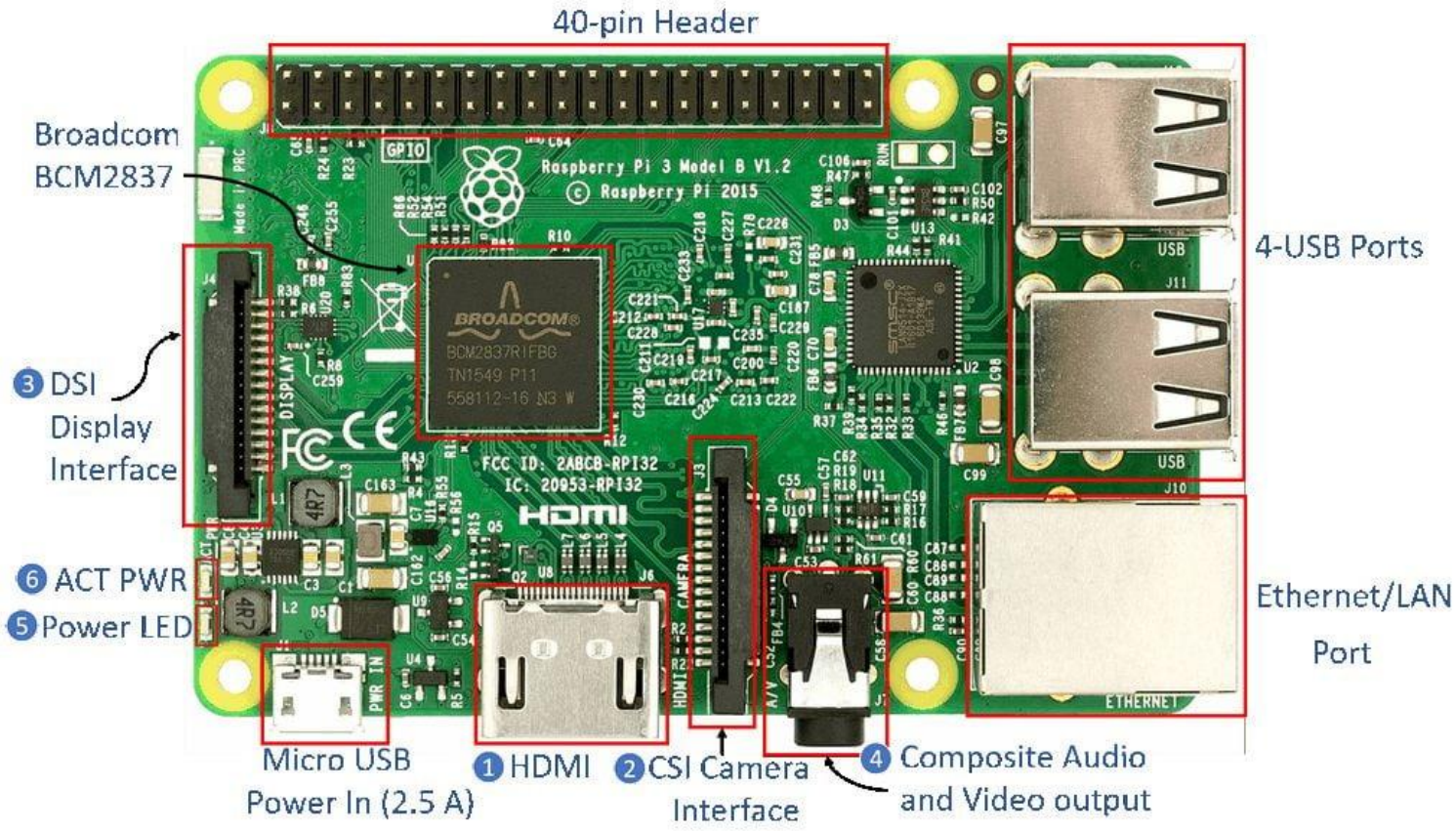
Introduction to Raspberry Pi

Why it is called as Computer ?

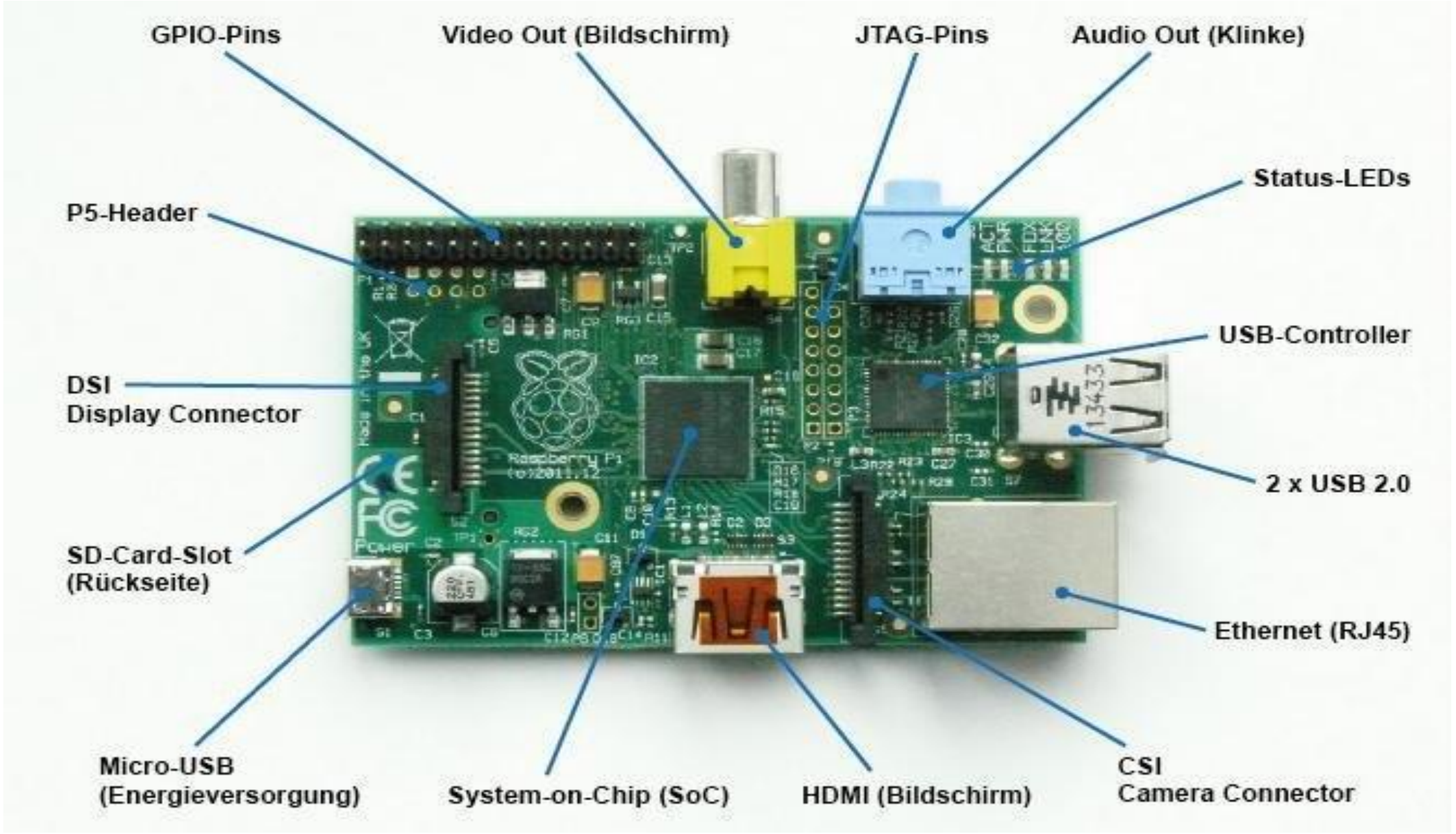
It runs with <u>processor</u>

Main Components of Raspberry Pi

- **Processor (CPU)** – ARM based
- **RAM**
- **GPIO Pins**
- **USB Ports**
- **HDMI Port**
- **Ethernet / Wi-Fi**
- **Micro SD Card** (acts as storage)



Introduction to Raspberry Pi



- **EXPLANATION ABOUT BOARD COMPONENTS**
- **1. System on Chip**
- CPU, GPU, SDRAM, and single USB port
- **2. CPU**
- A central processing unit is the brain of your Raspberry Pi. It is responsible for processing machine instructions, which are the result of your compiled programs.
- **3. GPU**
- The graphics processing unit (GPU) is a specialist chip designed to handle the complex mathematics required to render graphics.
- **4. USB 2.0 ports and 1 SoC on-board USB**
- The previous version of the Raspberry Pi Model B contained only a single microUSB port and a two standard USB ports. The Raspberry Pi 2 has been expanded to include an onboard 5-port USB hub.

- This allows you to connect four standard USB cables to the device and a single microUSB cable. The micro USB port can be used to power your Raspberry Pi 2.
- **5. SD card Slot**
- Raspberry Pi does not have a builtin operating a system and storage. You can Plug-in SD card loaded with a Linux image to the SD card slot.
- **6. DSI display connector –**
- used to attach an LCD panel. On the other side of the board is a **microSD card slot that holds the operating system.**
- **7. Camera Serial Interface(CSI)**
- This interface can be used to connect a camera module to Raspberry Pi.
- **8. Status LED's :**
- Raspberry Pi has 5 status LED's

Status LED	FUNCTIONS
ACT	SD card access
PWR	3.3 V power is present
FDX	Full Duplex Lan connected
LNK	Link/Network activity
100	100 Mbit Lan connected

9. Ethernet port

If you plan to place your Raspberry Pi near a router or switch or have enough Ethernet cable, then you can connect your Raspberry Pi directly with the Ethernet jack.

The Raspberry Pi 2 supports 10/100 Mbps Ethernet, and the USB adapter in the third/fourth port of USB hub can also be used for Ethernet via a USB to Ethernet adapter

10. GPIO pins

The main method for interacting with electronic components and expansion boards is through the **general purpose input/output (GPIO) pins on the Raspberry Pi.**

The Raspberry Pi 2 Model B contains 40 pins in total.

- GPIO pins can accept both input and output commands and can be controlled by programs in a variety of languages running on the Raspberry Pi.
- for example :The input could be readings from a temperature sensor, and the output a command to another device to switch an LED on or off.
- **11. HDMI Output :**
- This port provides both video and audio output.You can connect the Raspberry Pi to a monitor using HDMI cable .
- **12. Composite Video output :**
- Raspberry Pi comes with composite video output with RCA jack that supports both PAL and NTSC video output. RCA jack can be used to connect old televisions that have an RCA input only.
- **13. Audio Output :**
- Raspberrypi has 3.5mm audio output jack .This audio jacks use for providing audio output to old Televisions along with RCA jack for video .The audio quality from this jack is inferior to the HDMI output

Introduction to Raspberry Pi

Raspberry Pi Vs Arduino

Feature	Raspberry Pi	Arduino
Type	Single Board Computer (SBC)	Microcontroller Board
Processor	ARM-based processor (e.g., BCM2837)	Microcontroller (ATmega328, ATmega2560, etc.)
Operating System	Requires an OS (Raspberry Pi OS, Linux)	No OS required
Processing Power	High (supports multitasking)	Low to moderate
Programming	Python, C, C++, Java, etc.	C / C++ (Arduino IDE)
Boot Time	Takes time to boot	Starts instantly
Memory	RAM + SD card storage	Limited SRAM and Flash memory

Introduction to Raspberry Pi

Raspberry Pi Vs Arduino

Feature	Raspberry Pi	Arduino
Connectivity	USB, HDMI, Ethernet, Wi-Fi, Bluetooth	Serial, I2C, SPI (no HDMI)
GPIO Pins	40 GPIO pins (digital only)	Digital + Analog pins
Analog Input	Not supported (needs external ADC)	Built-in ADC available
Power Consumption	Higher	Very low
Cost	Comparatively higher	Cheaper
Best Used For	IoT gateways, image processing, servers	Sensor interfacing, real-time control
Real-Time Operation	Not real-time	Real-time

Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

- Raspberry Pi boards are grouped into different **series based on performance, size, and application focus**.

1. Flagship Series (Main / Standard Raspberry Pi Boards)

These are the **full-featured, high-performance boards** commonly used as small computers.

Characteristics

✓ Highest processing power
✓ Full-size board
✓ HDMI display support
✓ Multiple USB ports
✓ Ethernet + Wi-Fi + Bluetooth (Pi 3 onwards)
✓ Suitable for desktop & complex applications

Typical Applications

•General computing
•AI / ML at edge
•Robotics
•Multimedia systems

Introduction to Raspberry Pi

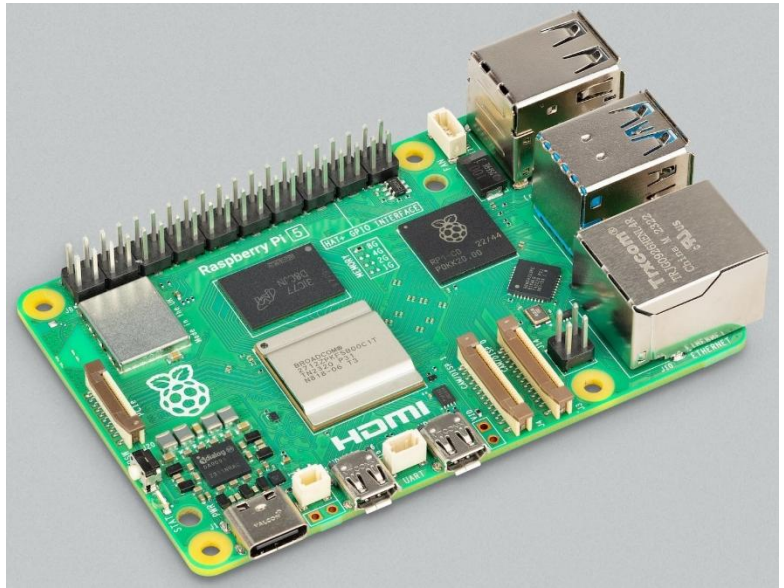
Raspberry Pi Product Series Classification

1. Flagship Series (Main / Standard Raspberry Pi Boards)

Examples

- Raspberry Pi 1 (Model A, B, A+, B+)
- Raspberry Pi 2 Model B
- Raspberry Pi 3 (3B, 3B+, 3A+)
- Raspberry Pi 4 Model B
- Raspberry Pi 5





Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

1. Flagship Series (Main / Standard Raspberry Pi Boards)

Versions

- ✓ Pi 1 → Basic & slow
- ✓ Pi 2 → Quad-core introduced
- ✓ Pi 3 → Wireless added
- ✓ Pi 4 → USB 3.0 + Dual Display + More RAM
- ✓ Pi 5 → Major speed & PCIe support

Models

- ✓ A → Basic, No Ethernet
- ✓ B → Ethernet + 2 USB
- ✓ B+ → 4 USB + 40 GPIO (Most Important)

Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

1. Flagship Series (Main / Standard Raspberry Pi Boards)

Feature	Raspberry Pi 1	Raspberry Pi 2	Raspberry Pi 3	Raspberry Pi 4	Raspberry Pi 5
CPU	Single-core ARM11 (700 MHz)	Quad-core Cortex-A7 (900 MHz)	Quad-core Cortex-A53 (1.2–1.4 GHz)	Quad-core Cortex-A72 (1.5 GHz)	Quad-core (Higher performance)
Performance	Low	Moderate	Good	Very High	Extremely High
RAM	256 MB / 512 MB	1 GB	1 GB	2 GB / 4 GB / 8 GB	Higher capacity & faster
Wi-Fi / Bluetooth	Not available	Not available	Available (Built-in)	Available	Available (Improved)

Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

1. Flagship Series (Main / Standard Raspberry Pi Boards)

Feature	**Raspberry Pi 1**	**Raspberry Pi 2**	**Raspberry Pi 3**	**Raspberry Pi 4**	**Raspberry Pi 5**
USB Ports	USB 2.0	USB 2.0	USB 2.0	**USB 3.0 + USB 2.0**	Faster I/O
Ethernet	10/100 Mbps	10/100 Mbps	Faster (B+)	**True Gigabit**	Improved
Display Output	HDMI	HDMI	HDMI	**Dual Micro-HDMI (4K)**	Enhanced graphics
Power Capability	Basic	Better	Better	High power demand	Optimized & higher
Typical Use	Learning & basics	Multitasking	IoT & wireless	Desktop & multimedia	High-end computing

Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

2. Zero Series (Compact & Low-Power Series)

Designed for **small size and low energy consumption**.

Characteristics

✓ Very small form factor
✓ Low power consumption
✓ Limited USB & I/O
✓ Lower cost
✓ Wireless connectivity in W models

Typical Applications

•Embedded systems
•IoT devices
•Wearables
•Portable electronics

Introduction to Raspberry Pi

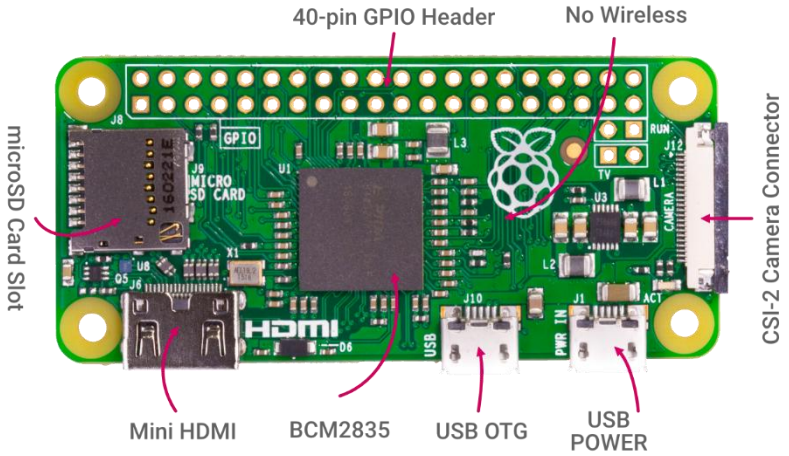
Raspberry Pi Product Series Classification

2. Zero Series (Compact & Low-Power Series)

Examples

- Raspberry Pi Zero
- Raspberry Pi Zero W (Wi-Fi + Bluetooth)
- Raspberry Pi Zero 2 W





Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

3. Compute Module Series (Industrial / Embedded Integration Series)

Specially designed for **commercial and industrial embedded products**.

Characteristics

✓ RAM & eMMC storage options
✓ No standard connectors (uses carrier board)
✓ Designed for custom hardware integration
✓ High reliability for products
✓ Flexible I/O configuration

Typical Applications

•Industrial automation
•Custom embedded products
•Digital signage
•Commercial devices

Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

3. Compute Module Series (Industrial / Embedded Integration Series)

Examples

- Compute Module 1 (CM1)
- Compute Module 3 / 3+
- Compute Module 4 (CM4)
- Compute Module 5 (latest)





Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

4. Pico Series (Microcontroller-Based Series)

Different from other Pi boards — **microcontroller, not a full computer**.

Characteristics

✓ Real-time control capability
✓ Very low power usage
✓ No OS required
✓ Programmable via MicroPython / C / C++
✓ Ideal for hardware interfacing

Typical Applications

- Sensor interfacing
- Control systems
- Embedded electronics
- IoT nodes

Introduction to Raspberry Pi

Raspberry Pi Product Series Classification

4. Pico Series (Microcontroller-Based Series)

Examples

- Raspberry Pi Pico
- Raspberry Pi Pico W (Wireless)

Core Chip

- **RP2040 microcontroller**

